



Department of Electrical and Computer Engineering,
Gina Cody School of Engineering and Computer Science, Concordia University,
Montreal, Canada

COEN 330 - Applied Machine Learning

Machine Failure Prediction Using the AI4I 2020 Predictive Maintenance Dataset

Group 12

Niraj Patel – 40211758

Samuel Lavallée 40284076

Thinoushan Senathirajah – 40251486

Omar Shrit – 40311810

Arnav Singh – 40258921

Instructor: Dr. Ahmed Bali

June 27, 2026

Table of Contents

<i>1. Abstract.....</i>	<i>3</i>
<i>2. Introduction and Motivation</i>	<i>3</i>
<i>3. Related Work or Background</i>	<i>3</i>
<i>4. Dataset Description</i>	<i>4</i>
<i>5. Preprocessing and Exploratory Data Analysis.....</i>	<i>5</i>
<i>6. Methodology and Models.....</i>	<i>7</i>
<i>7. Validation and Hyperparameter Tuning Strategy.....</i>	<i>7</i>
<i>8. Experimental Setup.....</i>	<i>8</i>
<i>9. Results and Model Comparison.....</i>	<i>9</i>
<i>10. Error Analysis and Qualitative Discussion.....</i>	<i>11</i>
<i>11. Demo or Usage Demonstration Description.....</i>	<i>12</i>
<i>12. Limitations and Future Work.....</i>	<i>12</i>
<i>13. Conclusion</i>	<i>12</i>
<i>14. Team Contributions</i>	<i>13</i>
<i>15. References.....</i>	<i>13</i>
<i>16. External Libraries Acknowledgment</i>	<i>13</i>

1. Abstract

This project develops a supervised machine learning workflow for predictive maintenance using the AI4I 2020 Predictive Maintenance Dataset. The task is binary classification: predict `machine_failure`, where 0 means no failure and 1 means machine failure. The inputs are product type, air temperature, process temperature, rotational speed, torque, and tool wear.

To avoid leakage, the diagnostic failure-mode columns TWF, HDF, PWF, OSF, and RNF were excluded from model training. Five classifiers were compared with a stratified 70/15/15 train/validation/test split. `HistGradientBoostingClassifier` was selected by validation F1-score, with recall, F2-score, and ROC-AUC used as tie-breakers.

On the held-out test set, the final model achieved accuracy 0.9853, precision 0.8718, recall 0.6667, F1-score 0.7556, F2-score 0.6996, and ROC-AUC 0.9750. Because failures are rare, this report emphasizes recall, F-scores, ROC-AUC, and the confusion matrix rather than accuracy alone.

2. Introduction and Motivation

Predictive maintenance uses machine operating data to estimate whether equipment is likely to fail. A useful model can help prioritize inspections and reduce unexpected downtime, repair costs, safety risks, and production delays.

This project frames the problem as supervised binary classification. The practical goal is to identify risky operating states using measurements available before diagnostic failure labels are known. The project therefore focuses on a complete applied ML workflow: clean the data, prevent leakage, compare models, tune hyperparameters on validation data, test once on held-out data, interpret the results, and provide a local demo.

The scope is intentionally limited. The AI4I dataset is synthetic and tabular, so the project demonstrates methodology rather than claiming production-ready industrial performance.

3. Related Work or Background

Predictive maintenance can be modeled as classification, regression, anomaly detection, forecasting, or survival analysis depending on the available labels. This project uses classification because the dataset provides a binary target indicating whether machine failure occurred.

For tabular predictive maintenance data, simple linear models provide useful baselines, while tree-based models can capture nonlinear feature interactions. This is relevant because failure risk may depend on combinations of torque, rotational speed, temperature, tool wear, and product type rather than any single feature alone.

4. Dataset Description

The AI4I 2020 Predictive Maintenance Dataset contains 10,000 synthetic observations. Each row represents one machine operating condition. The target column is `machine_failure`, and the positive class is machine failure.

Table 1. Dataset summary

Item	Description
Dataset name	AI4I 2020 Predictive Maintenance Dataset
Source	UCI Machine Learning Repository
Rows	10,000
Target	<code>machine_failure</code>
Task	Binary classification
Positive class	Machine failure
Limitation	Synthetic dataset, not direct factory data

Table 2. Model input features

Feature	Type	Description
<code>type</code>	Categorical	Product quality/type category
<code>air_temperature_k</code>	Numeric	Air temperature in kelvin
<code>process_temperature_k</code>	Numeric	Process temperature in kelvin
<code>rotational_speed_rpm</code>	Numeric	Rotational speed in revolutions per minute
<code>torque_nm</code>	Numeric	Torque in newton-metres
<code>tool_wear_min</code>	Numeric	Tool wear time in minutes

The positive class is rare: 339 rows are failures and 9,661 rows are no-failure cases. Failures therefore represent only 3.39% of the dataset. This imbalance affects both model selection and evaluation, because high accuracy can be achieved even when failures are missed.

Two identifier columns, UDI and Product ID, were removed because they are identifiers rather than machine-condition measurements. The diagnostic failure-mode columns TWF, HDF, PWF, OSF, and RNF were retained only for explanation and EDA. They were not used as model inputs because they directly describe failure causes and would leak information about the target.

The dataset is public and distributed through UCI under a CC BY 4.0 license. Because it is synthetic, there are no direct privacy concerns from real workers or equipment owners. The main limitation is that synthetic patterns may not perfectly represent real industrial machines.

5. Preprocessing and Exploratory Data Analysis

Preprocessing loads the raw CSV, cleans column names into snake_case, removes ID columns, preserves the binary target, and writes data/processed/ai4i_processed.csv. No missing values were found, so imputation was not required.

Model-specific transformations are kept inside the scikit-learn pipeline: type is one-hot encoded and numeric features are standardized. Keeping these fitted transformations inside the pipeline prevents validation or test information from leaking into training.

Table 3. Class balance counts

Class	Count	Percent
No failure (0)	9,661	96.61%
Failure (1)	339	3.39%

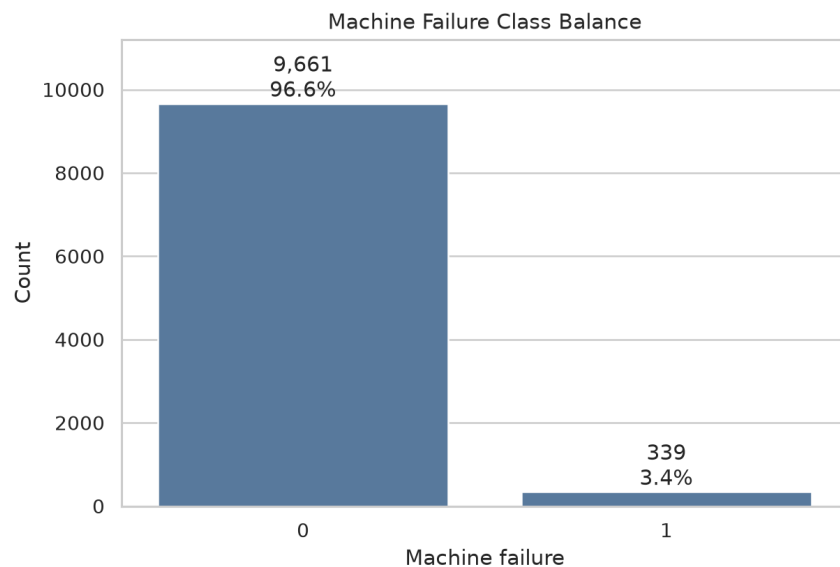


Figure 1. Class balance

Figure 1 shows the class imbalance. The no-failure class dominates the dataset, while the failure class is small. This explains why the evaluation cannot rely on accuracy alone.

Table 4. Classwise median feature comparison

Feature	No-failure median	Failure median	Difference
Torque (Nm)	39.9	53.7	+13.8
Tool wear (min)	107.0	165.0	+58.0
Rotational speed (rpm)	1507.0	1365.0	-142.0
Air temperature (K)	300.0	301.6	+1.6
Process temperature (K)	310.0	310.4	+0.4

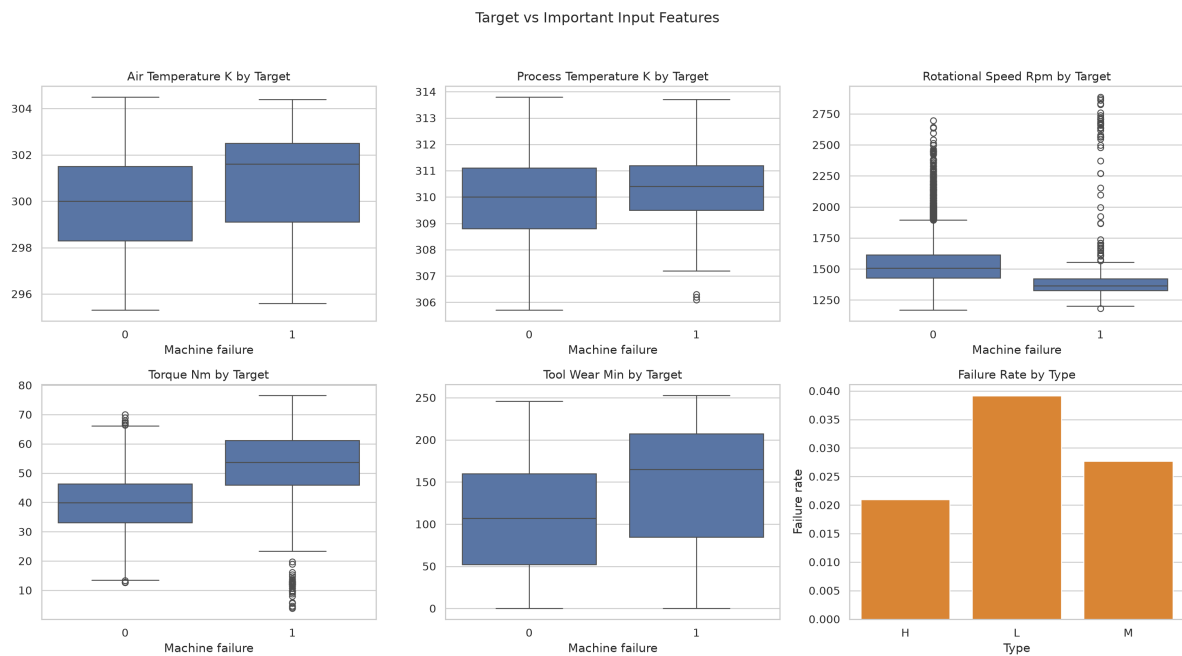


Figure 2. Target versus feature relationships

Figure 2 is the main EDA figure because it directly compares each input feature against the target. Failure cases have higher median torque, higher median tool wear, lower median rotational speed, and slightly higher temperatures. The type subplot shows that product type L has the highest failure rate. These patterns show that the target is related to the inputs, while the overlap between classes explains why the problem is not perfectly separable.

Table 5. Failure rate by product type

Type	Rows	Failure cases	Failure rate
H	1,003	21	2.09%
L	6,000	235	3.92%
M	2,997	83	2.77%

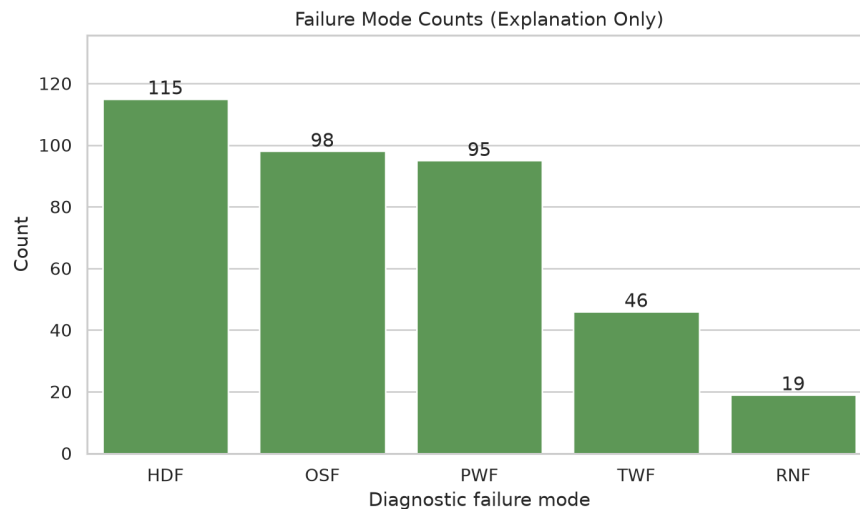


Figure 3. Failure mode counts

Figure 3 summarizes diagnostic failure-mode columns. HDF, OSF, and PWF are the most common diagnostic modes, followed by TWF and RNF. These columns help explain the dataset, but they are excluded from model training to avoid leakage.

6. Methodology and Models

The project compares five supervised classifiers to cover a range of model complexity: a simple baseline, an interpretable tree, two ensemble tree methods, and a boosted tree model.

Table 6. Candidate models and reasons for inclusion

Model	Role	Reason for inclusion
Logistic Regression	Baseline	Simple probability-based model used as a reference point.
Decision Tree	Interpretable nonlinear model	Uses rule-like feature splits and is easy to explain.
Random Forest	Tree ensemble	Combines many trees to reduce single-tree overfitting.
Extra Trees	Additional ensemble benchmark	Adds randomization to tree construction.
HistGradientBoostingClassifier	Boosted tree model	Strong nonlinear tabular model; selected final model.

Logistic Regression is the baseline because it is simple and interpretable. Decision Tree is included because it gives rule-like nonlinear splits. Random Forest and Extra Trees are ensemble methods that reduce reliance on one tree. HistGradientBoostingClassifier is a boosted tree model designed to perform well on tabular nonlinear data.

All models use the same approved input features and the same preprocessing pipeline. This keeps the comparison fair and ensures that training, validation, testing, and demo prediction use the same transformations.

7. Validation and Hyperparameter Tuning Strategy

The dataset is split into training, validation, and test sets using stratification and `random_state=42`. Stratification keeps the rare failure class represented in all splits. The validation set is used for model selection and hyperparameter tuning. The test set is reserved for final evaluation only.

Table 7. Train-validation-test split strategy

Split	Percentage	Purpose
Training	70%	Fits model parameters
Validation	15%	Tunes hyperparameters and selects model
Test	15%	Final held-out evaluation only

Three models were tuned: Decision Tree, Random Forest, and HistGradientBoostingClassifier. Logistic Regression and Extra Trees were kept as fixed comparison models. In total, 36 validation trials were recorded.

Table 8. Hyperparameter tuning trials

Model	Tuned hyperparameters	Trials
Logistic Regression	Fixed baseline settings	1
Decision Tree	criterion, max_depth, min_samples_leaf	18
Random Forest	n_estimators, max_depth, min_samples_leaf	8
Extra Trees	Fixed ensemble settings	1
HistGradientBoostingClassifier	learning_rate, max_iter, max_leaf_nodes	8

The final selected model was HistGradientBoostingClassifier with learning_rate=0.05, max_iter=100, and max_leaf_nodes=31. These hyperparameters were selected using validation performance, not the test set. After selection, the final model was refit on the combined training and validation data, then saved as models/final_model.joblib. The held-out test set was used only later in the evaluation script for final performance reporting.

The best model is selected by validation F1-score, with recall, F2-score, and ROC-AUC as tie-breakers. This is more appropriate than selecting by accuracy because missed failures matter more than a high overall correct rate on an imbalanced dataset.

8. Experimental Setup

The project is organized as a reproducible command-line workflow. The scripts handle preprocessing, EDA, training, evaluation, and demonstration. The training script saves the final pipeline as models/final_model.joblib, and the evaluation and demo scripts load that saved model instead of retraining it.

The saved model package includes the fitted preprocessing pipeline, selected model name, best hyperparameters, expected feature columns, target column, excluded leakage columns, random seed, and validation metrics. This makes the saved model usable and traceable.

Table 9. Reproducibility commands

Command	Purpose
python -m src.preprocessing	Create the processed dataset
python -m src.eda	Generate EDA tables and figures
python -m src.train	Train, validate, tune, select, and save the final model
python -m src.evaluate	Evaluate the saved model on the test split
python demo/demo.py	Load the saved model and run sample predictions

Table 10. Main project output files

Output file	Description
data/processed/ai4i_processed.csv	Cleaned dataset
models/final_model.joblib	Saved final pipeline and metadata
results/metrics_table.csv	Best validation result per model
results/hyperparameter_trials.csv	Complete validation trial history
results/test_metrics.csv	Final test metrics and confusion-matrix counts
results/plots/*.png	Final report figures

9. Results and Model Comparison

HistGradientBoostingClassifier achieved the best validation F1-score and was selected as the final model. Its validation performance showed the strongest balance between catching failures and avoiding unnecessary false alarms.

Table 11. Validation model comparison

Rank	Model	F1-score	Recall	F2-score	ROC-AUC
1	HistGradientBoostingClassifier	0.7742	0.7059	0.7317	0.9872
2	Decision Tree	0.7059	0.7059	0.7059	0.8478
3	Random Forest	0.6105	0.5686	0.5847	0.9766
4	Extra Trees	0.3939	0.2549	0.2968	0.9434
5	Logistic Regression	0.2298	0.7255	0.3895	0.8750

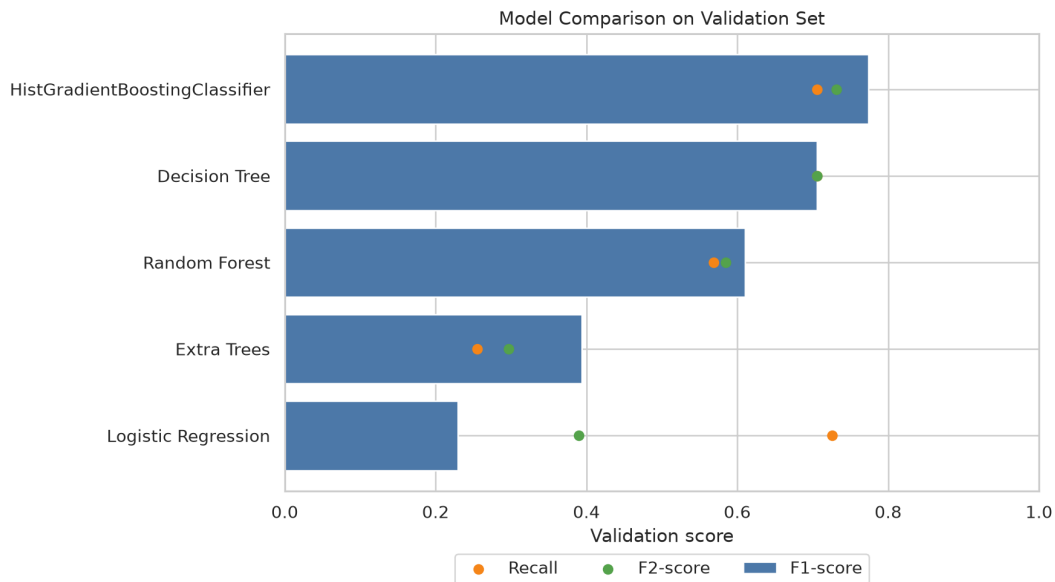


Figure 4. Model comparison on validation set

The model comparison figure shows the tradeoffs clearly. Logistic Regression had relatively high recall, but its low F1-score indicates many false positives. Extra Trees had weak recall. HistGradientBoostingClassifier had the best overall balance, so it was the most defensible final choice.

Table 12. Final test metrics

Metric	Value
Accuracy	0.9853
Precision	0.8718
Recall	0.6667
F1-score	0.7556
F2-score	0.6996
ROC-AUC	0.9750

Table 13. Final test confusion matrix counts

Actual class	Predicted no failure	Predicted failure
No failure	1444	5
Failure	17	34

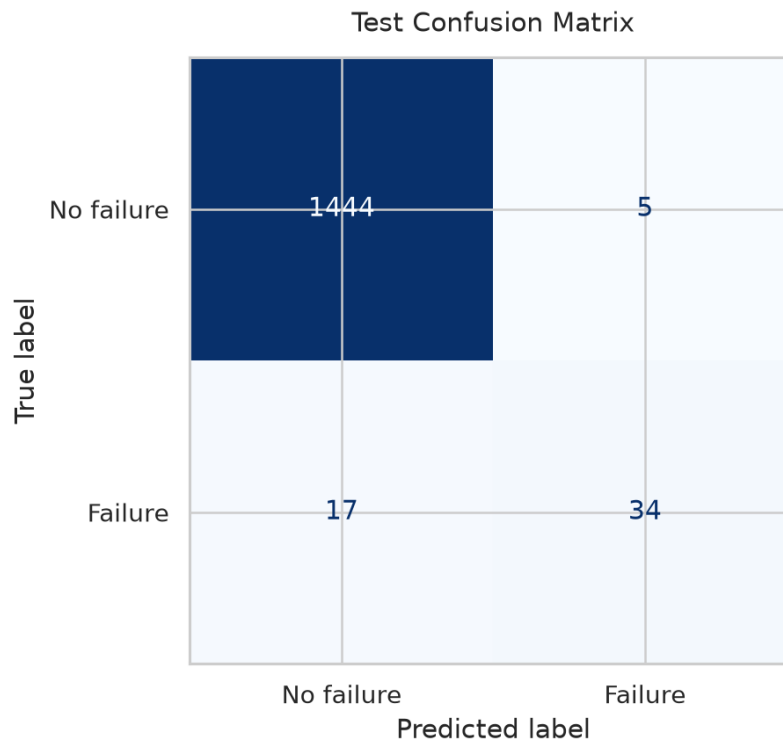


Figure 5. Test confusion matrix

The final test set contained 51 actual failures. The model caught 34 and missed 17, giving recall = $34 / 51 = 0.6667$. It also produced only 5 false positives among the no-failure cases. In maintenance terms, false positives are unnecessary inspections, while false negatives are missed failures. The model is therefore precise, but still conservative.

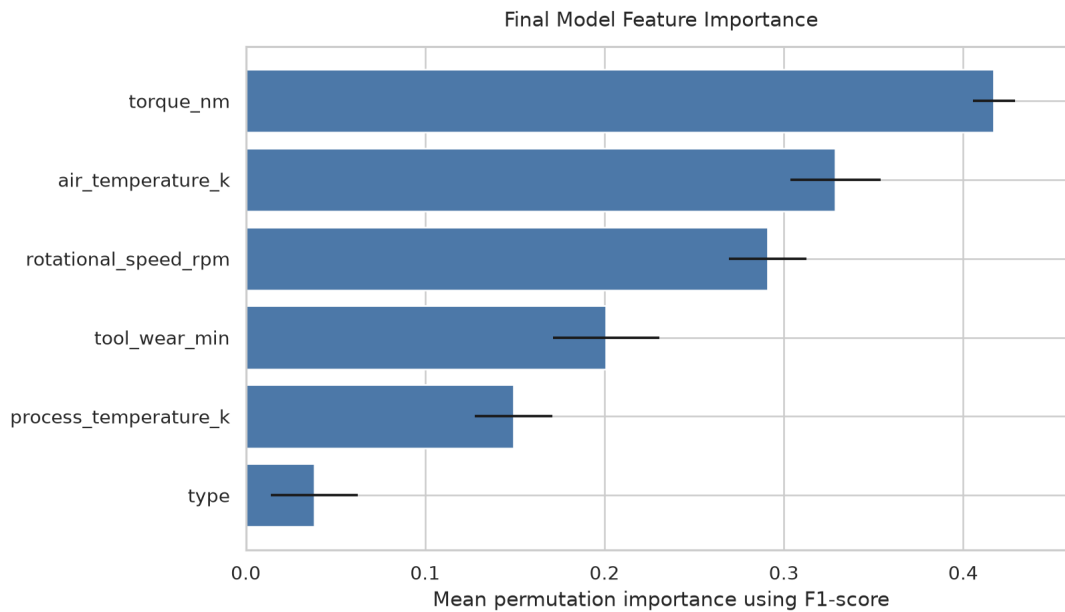


Figure 6. Final model feature importance

Permutation importance shows which features the final model relied on. Torque has the largest mean importance, followed by air temperature and rotational speed. The black error bars show variation across repeated shuffles; shorter bars indicate more stable estimates. The type feature has the lowest importance, meaning the continuous operating measurements were more influential than product type.

10. Error Analysis and Qualitative Discussion

The final model has high precision and strong ROC-AUC, but recall is lower than precision. This means the model is cautious: when it predicts failure, it is usually correct, but it still misses some true failures.

The 17 false negatives are the most important errors because they represent real failures predicted as safe. In a real maintenance setting, this could lead to missed inspections or unexpected downtime. The 5 false positives are less severe because they mainly represent unnecessary inspections.

The demo includes one incorrect prediction: processed row 77 is a true failure, but the model predicts no failure because its failure probability is 0.4113, below the default threshold. This example supports the recommendation for future threshold tuning. If missed failures are more costly than false alarms, lowering the threshold could improve recall, although it would likely increase false positives.

11. Demo or Usage Demonstration Description

The demo script `demo/demo.py` loads `models/final_model.joblib` and sample rows from `data/processed/ai4i_processed.csv`. It prints each sample row index, true class, predicted class, and predicted probability of failure.

The demo is intentionally minimal. It is not a production deployment, but it proves that the saved model can be loaded and used after training. The selected rows include both correct predictions and one missed failure so the demo supports both usage and error discussion.

Table 14. Demo sample predictions

Sample	Row index	True class	Predicted class	Failure probability
1	0	0	0	0.0005
2	1	0	0	0.0007
3	2	0	0	0.0005
4	50	1	1	0.8482
5	69	1	1	0.9579
6	77	1	0	0.4113

12. Limitations and Future Work

The first limitation is that the dataset is synthetic. Real industrial data may contain sensor noise, missing values, maintenance history, time dependence, and distribution shifts that are not captured here.

The second limitation is class imbalance. The project addresses this by using recall, F1-score, F2-score, ROC-AUC, and the confusion matrix, but a real system would need cost-sensitive evaluation and threshold selection based on maintenance priorities.

The third limitation is that the model uses the default classification threshold. Future work should tune this threshold, calibrate probabilities, compare precision-recall behavior, and validate performance on real predictive-maintenance datasets.

Additional future work could include SHAP or LIME interpretability, robustness checks under noisy data, a more polished dashboard, live sensor integration, model monitoring, and retraining procedures.

13. Conclusion

This project built an end-to-end supervised machine learning workflow for machine failure prediction. The workflow includes preprocessing, EDA, leakage prevention, model comparison, validation-based tuning, final test evaluation, feature interpretation, error analysis, and a local demo.

Five models were compared, including Logistic Regression as the baseline.

HistGradientBoostingClassifier was selected using validation performance, not the test set. On the held-out test set, it achieved strong accuracy, precision, F1-score, F2-score, and ROC-AUC, while catching 34 of 51 failures.

The model is useful as a course-project demonstration because it detects many failure cases while keeping false alarms low. However, missed failures remain important. Threshold tuning, cost-sensitive evaluation, and validation on real industrial data would be necessary before considering production use.

14. Team Contributions

Table 15. Team contributions

Team member	Contribution
Niraj Patel (40211758)	Coordinated the repository, integrated the end-to-end workflow, implemented preprocessing/training/evaluation scripts, prepared the demo, and assembled the reproducibility package.
Samuel Lavallée	Reviewed the dataset source, target definition, approved model inputs, leakage-column exclusion, and overall project consistency.
Thinoushan Senathirajah (40251486)	Reviewed EDA outputs, class-balance observations, target-versus-feature plots, plot usefulness, and feature-importance interpretation.
Omar Shrit (40311810)	Reviewed model comparison results, validation/test metrics, confusion matrix interpretation, and final model discussion.
Arnav Singh (40258921)	Reviewed final report structure, demo instructions, submission checklist, and final packaging requirements.

15. References

1. Stephan Matzka. AI4I 2020 Predictive Maintenance Dataset. UCI Machine Learning Repository. CC BY 4.0. <https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset>
2. Stephan Matzka. Explainable Artificial Intelligence for Predictive Maintenance Applications. 2020.
3. scikit-learn developers. scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
4. pandas development team. pandas documentation. <https://pandas.pydata.org/>
5. Matplotlib development team. Matplotlib documentation. <https://matplotlib.org/>
6. seaborn development team. seaborn statistical data visualization documentation. <https://seaborn.pydata.org/>

16. External Libraries Acknowledgment

This project uses standard Python libraries including pandas, scikit-learn, matplotlib, seaborn, joblib, and nbformat. The team reviewed and verified the final repository contents and remains responsible for understanding the submitted code, results, figures, and report.